Supporting Information: Emission Factor Recommendation for Life Cycle Assessments with Generative AI

Bharathan Balaji,[*,†] Fahimeh Ebrahimi,[†] Nina Gabrielle G Domingo,[†] Venkata Sai Gargeya Vunnava,[†] Abu-Zaher Faridee,[†] Soma Ramalingam,[†] Shikha Gupta,[†] Anran Wang,[†] Harsh Gupta,[†] Domenic Belcastro,[†] Kellen Axten,[†] Jeremie Hakian,[†] Jared Kramer,[†] Aravind Srinivasan,[‡] and Qingshi Tu[¶]

†*Amazon, USA*

‡*University of Maryland and Amazon Scholar, USA*

¶*The University of British Columbia, CA*

E-mail: bhabalaj@amazon.com

This document provides algorithm details, additional results, information about the prompts, NAICS codes, and the annotation process. It consists of **33 pages** and includes **2 figures**.

Algorithm details for EEIO LCA and process-based LCA

EEIO LCA

EEIO EF Dataset

We rely on the USEEIO dataset published by the US Environmental Protection Agency.[1] The USEEIO assigns an emission factor to unique industry sector codes as defined by the U.S. Bureau of Economic Analysis (BEA) and the North American Industry Classification System (NAICS). The NAICS codes, published by the United States Census Bureau[2], represent detailed industry sectors, with the 6-digit codes organized hierarchically under 4-digit and 2-digit levels. For instance, the NAICS codes for chocolate and candy making (311320) and crystallized fruits making (311340) map to the 4-digit code for sugar and confectionery product manufacturing (3113). Each BEA code roughly corresponds to a 4-digit NAICS code, and the USEEIO EFs are assigned at this level in units of kilogram CO_2 equivalents per \$1 ($kgCO_2e/\$$) of commodities produced by the industries. We use USEEIO version 2.1 dataset for our experiments, which corresponds to NAICS sectors defined in 2017.

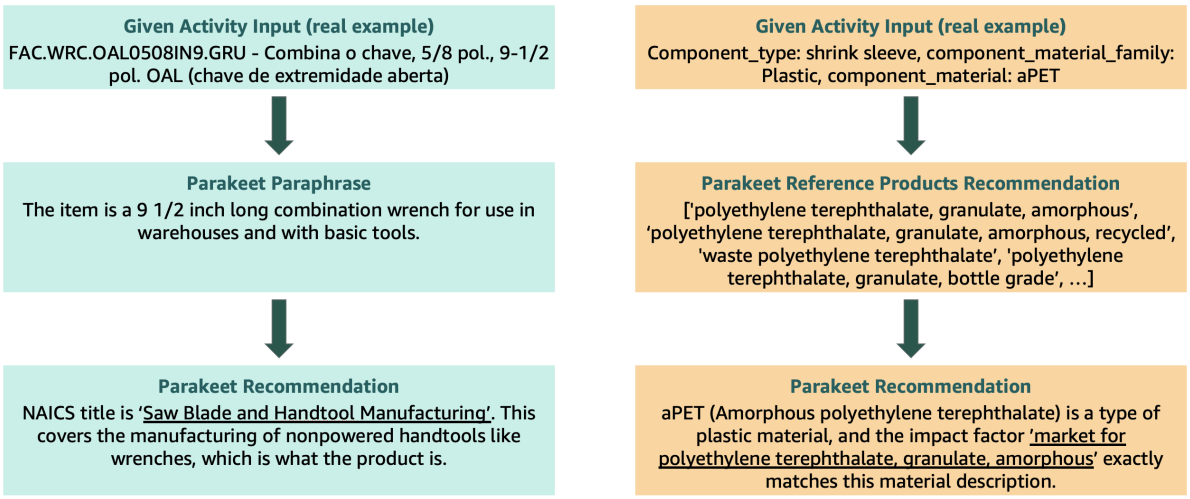
There are 1016 NAICS sectors which are grouped into 411 BEA industry sectors, with an EF associated at each BEA sector in the USEEIO dataset. However, each NAICS sector can encompass multiple related industries. For example, the industry sector "Saw Blade and Handtool Manufacturing" consists of 111 industries, such as manufacturing of wrenches, axes, saw blades, etc. Embedding models do not capture such set relationships, and catch-all industry sectors with the term "miscellaneous" are ill-defined. Following Balaji et al.[3], we perform semantic text search on the detailed industry descriptions instead of the NAICS sector titles as it improves retrieval relevance by >20%. There are a total of 19723 industry descriptions in the NAICS documentation.

EEIO LCA EF recommendation

EEIO LCA gives the first-order approximation of the carbon footprint, when no impact information is available about a given product or service. To get an estimate, all we need is a description of the business activity and the money spent on it. Such data is tied to enterprise resource planning systems, such as invoices and procurement. We find that the text from these sources often contains finance-specific terms (e.g., COGS – cost of goods sold), domain-specific terms (e.g., AMZL for Amazon's shipping services), and other abbreviations (e.g., PCS for pieces). Therefore, using the LLM to create a plain text description of the query significantly enhances the ability to retrieve semantically meaningful results with an embedding model.

While embedding models are good at retrieving the relevant EFs, the top-ranked EF sorted by cosine similarity may not yield the best answer. For example, to select the right EF, one needs to identify the industry sector that captures the GHG emissions associated with a given product. A "wireless access point" can belong to installation contractors, internet service, or wireless equipment manufacturing. From an emissions point of view, the equipment manufacturing sector is the correct choice. To mitigate such issues, we use the LLM to recommend the best matching EF, and incorporate such instructions into the prompt. Figure 1a shows an example of a combination wrench mapped to the Saw Blade and Handtool Manufacturing sector.

We provide the human annotators with the top EF recommendation along with its justification, as well as the top-10 EF shortlisted via semantic text matching. If the annotator agrees with the top recommendation, then Precision@1 for that query will be correct. If the human disagrees with the top recommendation, but picks an EF from the top-10 short-list, then Precision@10 is considered correct but Precision@1 is incorrect. If the human says none of the EFs are appropriate and that the given query is clear, the algorithm's output is considered incorrect. We filter the queries that annotators mark as unclear or 'not sure' for computing the performance metrics.



(a) EEIO LCA **(b)** Process LCA

Figure 1: Two examples of Parakeet mapping product descriptions to EFs. Figure (a) shows Parakeet suggesting the product is a 'combination wrench' from the given activity description and using that it can recommend the most appropriate NAICS title (EEIO LCA). Figure (b) shows an analogous example for process LCA where Parakeet finds the closest reference product and associated impact factor for a product it paraphrases as 'shrink sleeve' from the given activity description.

Process LCA

Process LCA EF Dataset

We use the Ecoinvent v3.9 (cut-off system model) dataset,[4] a widely-used source of background dataset compiled by a non-profit association. We compute the corresponding EFs using the IPCC AR6 impact assessment for global warming potential over 100 years method with the unit of $kgCO_2e$ per unit reference product. The resulting dataset contains ∼21k EFs for ∼3.5K unique reference products, covering a broad range of sectors like agriculture, construction, and chemicals. The EFs within Ecoinvent represent average production systems within their corresponding geographic location, rather than company or site specific production. We ignore the location information in our study, and end up with ∼9.3K unique EFs.

Process LCA EF recommendation

In case of process LCA, the query data consists of detailed information about a particular product such as its bill of materials, manufacturing process, and transportation modes.

The EF datasets contain highly technical terms such as drip irrigation and steel trawler maintenance. Each EF is specific to a particular product, and some EFs can be missing in the dataset, e.g., while there are EFs for tilapia and trout in the Ecoinvent database, there are no EFs for salmon. For a human, identifying the EF is challenging as they need to understand domain-specific terms, and ensure their search is exhaustive to ensure they do not miss an available EF. For example, eggplant is referred to as "aubergine" in Ecoinvent, and a user may miss it if they are not aware of that synonym.

The definition of 'match' and 'no match' is critical in determining when the EF recommendation is correct. We deem the recommendation to be a 'match' only when the system boundary of the EF exactly matches the given input. We consider 'trout' EF to be not a match for 'salmon' as there is no data to compare the processes used to produce salmon to that of trout. Similarly, we do not consider 'wheat flour' to be a good match for 'bread' as it excludes other ingredients as well as the baking process. We choose this definition of 'match' to create an objective criteria, while LCA experts in practice can make different choices based on data available and the goals of their study. Even when Parakeet outputs 'no match', it includes the closest matching EFs for the given query. So the algorithm is still useful to the expert, when it is used with a human in the loop.

Given the definition of 'match', it is critical to identify when no EF in the dataset matches a given query. Simply using distance thresholds with semantic text similarity fails to identify when no EF in the dataset is a good match.[5] Balaji et al.[6] use intermediate classification layers to improve the "no match" precision with semantic text matching, where the an EF is considered an appropriate match only when both the query and EF belong to the same industry sector. In contrast, we leverage LLMs to determine whether an appropriate match exists from the ranked list of semantically matched EFs. Using prompts, we provide additional context and incorporate expert knowledge to guide the LLM in EF recommendation.

We use a two stage matching solution for process EF recommendation in Parakeet. We leverage the metadata called "reference product" in the Ecoinvent dataset, which captures

the high-level description of the product the EF is referring to. For example, there are separate EFs for aubergine grown in an open-field versus a greenhouse, but the reference product for both is "aubergine". The reference product metadata is also available in other large datasets such as GaBi[7], and when its not available an industry classification system such as ISIC (International Standard Industrial Classification) or CPC (Central Product Classification) can be used instead. Given a query, we first identify the closest reference product, and then search for an appropriate EF that belong to the reference product.

We identify the appropriate reference products using the same steps as Figure **??**, where EFs are replaced by reference products. In this step, instead of requesting one reference product, we asked the LLM to recommend a list of appropriate reference products. We pass all the EFs that belong to retrieved reference products to the LLM, and ask it to pick the best matching EF. The prompt includes the full detailed description of processes included in the EF so that LLM has sufficient context for recommendation. The LLM is instructed to output "no match" with an appropriate justification when no EFs are relevant to the given query. Figure 1b shows an example of a 'shrink sleeve' being mapped to 'market for polyethylene terephthalate'. We also include a figure of Parakeet pLCA recommendation algorithm in Supplemental Information (SI).

In terms of human annotation, we follow the same guidelines as EEIO-LCA EF recommendation. For the queries that have do not have a matching EF in the given dataset, we deem the algorithm to be Precision@1 correct when the top recommendation is "no match". The algorithm is incorrect when it recommends an EF and human selects "no match" as the appropriate response. For Precision@10, the algorithm is correct if the annotator picks one of the short-listed EFs as an appropriate choice, and incorrect when annotator picks "no match", but the algorithm recommended an EF with a justification.

Additional Results

Ablation Analysis

To understand the contribution of each component of Parakeet, we do an ablation study where we selectively remove steps in the algorithm and measure the impact on performance. In Table 1, we show the results of the ablation on EEIO-LCA with the KatanaML dataset. Removing paraphrasing, which translates the given input text to plain English, reduces Precision@1 by 11.4%. If we remove the final EF recommendation by the LLM, performance drops drastically by 56.9%. If we remove the semantic matching step, and only use LLMs for both paraphrasing and EF recommendation, we also see a large drop in performance (40.1%). Therefore, use of both LLM and semantic model in tandem is essential to get high Precision@1 values. LLMs are good at picking the right EF given the short-list from semantic matching. Without semantic matching, LLM is given all the EFs as input, and it is unable to pick the correct EF consistently. It also increases the size of the input, which increases the latency of the output by 1.7x. Note that the semantic matching solution in table corresponds to the prior state-of-the-art,[3] and Parakeet outperforms the algorithm significantly. On the KatanaML dataset, CaML achieved a Precision@1 of 31.5% and Precision@10 of 99.3% for EEIO-LCA tasks. In comparison, Parakeet achieves a Precision@1 of 97.1% and Precision@10 of 100%, demonstrating a substantial improvement of 65.6% in terms of Precision@1."

Table 1: An ablation study demonstrating the performance contributions (Precision{@1, 10}, MAPE, latency) of each component of Parakeet on the KatanaML dataset.

Ablation	Precision@1	Precision@10	MAPE	Latency
Default	**97.1**	100	1.6	11.2s
Remove paraphrasing	85.7	99.4	7.5	9.1s
Remove LLM recommendation	40.2	99.5	27.3	1.4s
Semantic matching, no LLM[3]	31.5	99.3	32.1	0.1s
No semantic matching	57.0	57.0	34.1	19.5s

Another interesting observation is that performance remains high in terms of Preci-

sion@10, except for the last case where no semantic matching is used. Hence, in a human-in-the-loop system, it might be sufficient to just do semantic match with a latency of just 0.1s, a speedup of >100x. However, we do find that humans can get confused given closely related options and LLM recommendation helps them make the right choice when there is ambiguity. For instance, when given a complex input like "FAC.WRC.OAL0508IN9.GRU — Combina o Chave" (which contains abbreviations and multiple languages), the LLM correctly paraphrased it as a "combination wrench" and recommended "Saw Blade and Handtool Manufacturing" as the best matched impact factor. This example demonstrates how the LLM can help users interpret complex queries that would typically require multiple steps for manual identification. Similarly, Parakeet correctly identified "Little Boys Size 2 Son Of Mars" as a shoe and recommended "Footwear Manufacturing", recognized "Master and God" and "A Woman of Substance" as books, and decoded "Case Erecter" with category "Autobox Machines" as "Packaging Machinery". In each instance, the LLM provides reasoning for its choice. For example, in the case of packaging machinery, it explained: "... the most appropriate NAICS title match is 'Packaging Machinery Manufacturing'. This category includes the manufacturing of packaging machinery, which aligns with a case erector machine used with auto box machines to erect cardboard cases. The other NAICS titles relate to different types of boxes, containers, machinery, and equipment not specifically used for packaging operations." Navigating the balance between relying on human expertise versus LLM recommendation can be challenging in practice. In cases where we need to ensure EFs are chosen with high-precision, we ask multiple humans to make a selection and consider the majority vote as the correct answer. Future works can explore using multiple LLMs to improve precision as well as output a confidence score to trigger human evaluation.

In Table 2, we ablate the steps in the pLCA EF recommendation for the Food.com ingredients dataset. We find that LLM-based paraphrasing of the input reduces Precision as the model focuses on the primary component of the input while overlooking or losing information about its different constituents. An example of this type of error is the model

Table 2: An ablation study demonstrating the performance contributions (Precision{@1, 10}, latency) of each component of Parakeet in recommending pLCA Impact Factors in Food.com Ingredients dataset (2K samples).

Ablation	Precision@1	Precision@10	Match Precision	MAPE	Latency
Default	71.0	72.9	98.1	1.0	17.1s
Add Paraphrasing	59.7	64.1	79.8	98.1	19.3s
No LLM Reference Product	70.0	73.7	90.1	8.6	8.9s
No Semantic Matching	62.5	80.5	0.0	9317197.7	43.3s
Semantic match only[8]	0.0	18.1	0.0	354482.4	0.2s

(with paraphrasing) selecting "market for spinach" for the item "canned spinach". In this case, the actual EF should have been "no match" because the "canned spinach" item involves canning and processing of the spinach, which the model failed to fully account for. Note that the Match Precision values also drop by 18.3%, indicating that improper paraphrasing also interferes with selecting the correct EF. We hypothesize that better paraphrasing instructions to the LLM can overcome this limitation.

In pLCA, Parakeet first short-lists the reference product that the input belongs to before selecting the final EF. If we remove the reference product step, then performance of Match Precision drops by 8%, indicating that having a reference products step helps to narrow the type of EFs that need to be considered in the final selection. Note that prior works recommended EFs at the reference product level, and averaged the EFs belonging to a single reference product.[6,9] Therefore, the prior state-of-the-art algorithm that used semantic matching drastically reduces in performance due to the specificity of the EF.[6,8] When we measured the performance of semantic match with reference products prediction, the Precision@1, Precision@10, and Match Precision improved to 11.3%, 100%, and 62.4%.

We evaluated the Flamingo algorithm[6] on the Food.com Ingredient dataset. Flamingo uses intermediate classification layers to improve the "no match" precision with semantic text matching. We assessed the model's precision by comparing its recommended reference products against the collected ground truth data. The algorithm achieved Precision@1 of 76.6%, Precision@10 of 77.2%, and Match Precision of 78.8% , with a no-match precision of

65.2%. In comparison, Parakeet achieved Precision@1 of 71.2%, Precision@10 of 72.9%, and Match Precision of 90.4%, with a No-match precision of 54.9%. Flamingo shows better performance in identifying cases where no suitable match exists, while Parakeet demonstrates superior accuracy in identifying the correct reference product when matches are available. These results suggest that Flamingo is more effective at recognizing when a product doesn't have a match in the database, while Parakeet is more precise at identifying specific emission factors when matches do exist. This complementary performance, combined with Parakeet's human-readable justifications, makes it particularly effective when used with human verification for final EF selection.

Choice of LLM

We examine how the choice of the LLM impacts the performance of Parakeet. Tables 3 and 4 study the impact on the Katana ML invoices and Food.com ingredients datasets respectively.

In case of EEIO-LCA with KatanaML invoice dataset, the results are largely in line with expectation from external benchmarks where the larger LLMs such as Claude 3 Sonnet outperform smaller models such as Claude 3 Haiku and Llama 3.1 Instruct 70B. Claude 3 Haiku, in particular, is $\sim$2.9x faster than Claude 3 Sonnet, and has almost the same performance as Claude 3 Sonnet in terms of Precision@10. Therefore, for human-in-the-loop systems, it may be a better choice as the human can make the final selection from the recommended 5 EFs. An anomaly in the experiment is that Claude 3 Sonnet outperforms Claude 3.5 Sonnet, which is not the case in most external benchmarks. Upon analysis, we discovered that the differences in Claude 3 and 3.5 Sonnet models were primarily due to ambiguity in input or the instructions provided. For example, the query 'The Puzzle Palace' was correctly assigned to 'Book Publishers' by Claude 3 Sonnet, but Claude 3.5 Sonnet assigned it to 'Toy and Hobby Goods and Supplies Merchant Wholesalers'. For the input 'The Epic of Gilgamesh', Claude 3 Sonnet chose 'Book Publishers' while Claude 3.5 Sonnet chose 'Book Printing'. In some cases, the recommendation of Claude 3.5 Sonnet identified

errors in the annotated ground truth. For example, the model correctly identified 'Shoeless Joe' and 'Tea with Jane Austen' as books rather than shoes or tea by Claude 3 Sonnet. These discrepancies indicate the need for more precise instructions, and the importance of working with the latest models that learn from the latest knowledge on the web. Collecting ground truth at scale is another challenge as model predictions can bias human selection.

Table 3: Comparison of Parakeet's performance (Precision{@1, @10}, Latency) with different LLMs in recommending EEIO Environmental Factor (EF) in KatanaML dataset.

Large Language Model	Precision@1	Precision@10	MAPE	Latency
Llama 3.1 Instruct 70B	81.3	96.8	15.5	37.8s
Claude Instant 1.2	76.8	95.3	11.9	6.9s
Mistral Large	82.3	94.8	15.2	12.7s
Claude 3 Haiku	80.4	99.6	11.8	**3.9s**
Claude 3 Sonnet	**97.1**	**100**	1.6	11.2s
Claude 3.5 Sonnet	88.6	99.9	7.2	10.2s

Table 4: Comparison of Parakeet's performance (Precision{@1, @10, Match Precision}, Latency) with different LLMs in recommending pLCA Impact Factors in Food.com Ingredients dataset (2K samples).

Large Language Model	Precision@1	Precision@10	Match Precision	MAPE	Latency
Llama 3.1 Instruct 70B	80.7	84.8	96.4	2.1	14.9s
Claude Instant 1.2	70.1	77.6	69.8	37.6	**7.2s**
Mistral Large	78.8	83.3	**98.1**	21.7	10.6s
Claude 3 Haiku	71.5	81.2	97.0	24.6	13.2s
Claude 3 Sonnet	71.0	72.9	**98.1**	1.0	17.1s
Claude 3.5 Sonnet	**81.4**	**87.6**	97.0	3.6	21.5s

In case of pLCA EF recommendation for the Food.com dataset, we add another performance metric, called 'Match Precision'. About 82% of the data points do not have a matching EF, and the results can vary significantly depending on the LLMs to predict 'No Match'. From Table 4 we can see that the majority of the LLMs have high Match Precision (>95%). The newer LLMs such as Claude 3.5 Sonnet and Mistral Large outperform the previous generation of LLMs such as Claude 3 Sonnet. The improve performance is expected as the latest models have improved reasoning capabilities and are trained on larger scale web data.[10] Overall, Claude 3.5 Sonnet gives the best performance, and Mistral Large provides

the best performance to latency trade-off.

Numbers of Examples in Prompts

We investigate the effect of varying the number of examples on the performance of Parakeet, utilizing Claude 3 Sonnet. Tables 5 and 6 present the results for the KatanaML invoices and Food.com datasets, respectively. We adjust the number of examples in all prompts (paraphrasing, reference product selection, and EF recommendation) from zero to three. In ML literature, these are referred to zero-shot to three-shot prompts.[11]

Table 5: Effect of prompting strategies in Parakeet's performance (Precision{@1, 10}, latency). Demonstrated on KatanaML dataset.

Prompt Type	Precision@1	Precision@10	MAPE	Latency
Default (one-shot)	**97.1**	**100**	1.6	11.2s
Zero-shot	87.1	99.7	5.5	**10.9s**
Three-shots	91.3	99.9	4.7	13.7s

As shown in Table 5, for KatanaML invoices dataset, including a single example in the prompts yielded the best overall performance, with a Precision@1 of 97.1%. The significance of this single example became evident when its removal resulted in a decrease of Precision@1 to 87.1%. Increasing the number of examples to three decreased the Precision@1 by nearly 6% for the KatanaML invoices dataset. This decline might be attributed to the increased prompt length, which caused the model to deviate from the given instructions.

For the pLCA Food.com dataset, removing examples from the prompt increased Precision@1 by nearly 11%. However, the Match Precision decreased by 8%. This indicates that having at least one example improves performance when there is an EF matching the given input. Three-shot prompts resulted in lower performance, although the Match Precision remained consistent with the default. With three examples, the model's errors were primarily due to recommending an EF when the correct answer was 'no match'. One possible explanation for this could be the increased prompt length due to additional examples, which may have caused the model to overlook the instruction about recommending an 'exact

match'. Another factor could be the types of examples included. Specifically, two out of three examples had an exact match, while only one was a 'no match' case.

Table 6: Effect of prompting strategies in Parakeet's performance (Precision{@1, 10}, latency). Demonstrated on Food.com Ingredients dataset (2K samples).

Prompt Type	Precision@1	Precision@10	Match Precision	MAPE	Latency
Default (One-shot)	71.0	72.9	**98.1**	1	17.1s
Choose EF: Zero-shot	**82.8**	**88.5**	90.4	9.9	**12.2s**
Choose EF: Three-shot	56.9	58.1	97.1	22.6	18.3s

Choice of Embedding Model

Table 7: Comparison of Parakeet's Performance (Precision@1, Precision@10) of different Embedding Models used to derive and rank cosine similarities between economic activity (query) and EFs in KatanaML dataset.

Embedding Model	Precision@1	Precision@10
all-mpnet-base-v2	77.7	98.3
thenlper/gte-large	**97.1**	**100**
BAAI/bge-large-en-v1.5	86.3	99.8
Cohere	87.6	99.8
intfloat/multilingual-e5-large-instruct	81.5	99.5
avsolatorio/GIST-large-Embedding-v0	93.4	99.7

Table 8: Comparison of Parakeet's Performance (Precision{@1, 10}) of different Embedding Models used to derive and rank cosine similarities between economic activity (query) and EFs, demonstrated on Food.com Ingredient dataset (2K samples).

Embedding Model	Precision@1	Precision@10
all-mpnet-base-v2	69.8	**74.6**
thenlper/gte-large	**71.0**	72.9
BAAI/bge-large-en-v1.5	66.8	70.3
Cohere	68.3	72.9
intfloat/multilingual-e5-large-instruct	68.6	71.8
avsolatorio/GIST-large-Embedding-v0	67.9	70.6

We evaluate the performance impact on Parakeet with different embedding models. Tables 7 and 8 show the results for EEIO-LCA and pLCA respectively. Overall, the choice of the embedding model does not have a significant impact on performance. The Precision@10

is ∼100 for nearly all models in EEIO-LCA, and both Precision@1 and Precision@10 varies within a few percentage points for pLCA. However, there is a significant variation in Precision@1 performance for EEIO-LCA, which indicates that the ranked order of EFs presented to LLMs has an impact on the final recommended EF. The performance of the embedding models are in-line with external embedding benchmarks,[12] with `thenlper/gte-large` being the best model among those we evaluated.

The embedding models we have evaluated have much lower latency compared to the LLMs (0.2s vs 16.9s). However, state-of-the-art embedding models can be slower, larger, and therefore, more expensive. As the EF datasets become larger, the latency of embedding models can be a significant factor in the final choice for real-world applications.

Extension to Location Information

The principles of our method can be extended to account for region information when it is available as input. To show this, we have enhanced Parakeet to incorporate location-specific considerations when suggesting EFs. This improvement affects both pLCA and EEIO analyses.

For pLCA, we modified the LLM instructions to include location-based logic. The system now evaluates geographic information from the input data when selecting impact factors. When multiple candidates meet the exact match criteria, the algorithm selects the factor whose geographic scope best aligns with the input location. In cases without specified locations, the system defaults to 'Global' scope factors or those with the broadest geographic coverage. The selection process prioritizes factors that best represent greenhouse gas emissions from raw materials and manufacturing, followed by geographic relevance.

For EIO analysis, we switched from USEEIO EFs to multi-regional input-output (MRIO) EFs. These MRIO factors were derived from EXIOBASE3 Input-Output Tables[13] and NAICS to EXIOBASE sector mapping concordances.[14] The enhanced recommendation algorithm now employs a two-step process: first, it uses an LLM prompt to identify the most

appropriate region for the activity location (defaulting to 'United States' when unspecified); second, it maintains Parakeet's original functionality by selecting the most suitable NAICS code and title matching the input. The final recommendation combines both the location-specific and NAICS code selections.

To evaluate Parakeet in recommending region specific EFs, we use a proprietary dataset of 1.5k unique activities from 123 retail product life cycle inventories. The model's outputs were annotated by three human annotators and the final EF was calculated using majority voting. For pLCA EFs, the modified algorithm achieved 58.3% precision@1 and 99% precision@5 for reference product matching. For EIO EFs, it achieved 94% precision@1 and 99% precision@5 for NAICS code matching and 99% accuracy in location selection. Analysis of pLCA errors revealed that 61% of the errors resulted from selecting "production" over "market" EFs, possibly due to the increased length of the prompt with location-based instructions. Another 27% of the errors stemmed from insufficient life cycle stage context (e.g., truck transportation activity data was mapped to truck production emission factor), while the remaining errors split between incorrect location selection (6%) and valid but different EF choices between Parakeet and annotators (e.g., Parakeet and annotator chose different neighboring countries as a proxy for the target location).

Prompts

This section presents all prompts we used in the Parakeet algorithm.

LCA assistant prompt

```
The following is a conversation between a life cycle assessment (LCA) expert
and an LCA AI assistant.


Current conversation:
```

```
{history}


Human:
{input}


Assistant:
```

Paraphrase prompt

```
I want to do LCA of business activities based on Environmentally
Extended Input Output (EEIO)  Environmental Impact Factors (EIF).
I'm interested in the environmental impact associated with the materials
and manufacturing phase of the activity. I am given business activity
descriptions, and I want to paraphrase it to a plain language description
before I select an EIF.

Below is an example of a given activity and its plain language description.
Note that the description  is brief, and does not make any assumptions
about the activity.

itemname                 PRT.CLR.MFP.LRG - HP Color LaserJet Ent MFP M5...
itemdescription                                            PRT.CLR.MFP.LRG
itemcommodityname                                                 Printers
unique\_identifiername                                  UIN - OPERATIONS IT

The item is a color laserjet printer used for operations in an
IT environment.
```

```
Following the example, provide a plain language description of the activity
data given below:
{}

Make the most of the given information. DO NOT say that information is
limited. DO NOT refrain from providing a description, or ask for more
information. If you cannot provide a plain language description, simply
summarize the information provided. You MUST provide a description.

Avoid filler words such as "Based on the details" or "happy to assist",
keep your response to the point. Do not repeat the given instructions or
information.  DO NOT say you have insufficient information for an LCA.

Only provide the description and nothing else.
```

Recommend the best NAICS sectors prompt

```
### Instructions ###
You want to perform economic input-output life-cycle assessment, or EIO-LCA.
You have description of an activity and a few relevant North American
Industry Classification System (NAICS) titles, their descriptions and codes.
Reorder these NAICS titles and codes from the most likely relevance to the
least likely relevance. Only report the top 5 unique titles and associated
codes.
The criteria for relevance is the environmental impact associated with
the materials and manufacturing phase of the activity.
Be sure to properly escape any special characters in the ‘justification‘
```

field so that your response can be parsed with a python code interpreter. Return a Python list of dictionaries where each dictionary contains the keys ‘naics_code’, ‘justification’, and ‘naics_title’. The format of each dictionary must match the following example: [{{‘naics_code’: , ‘justification’: ‘’, ‘naics_title’: ‘’}]. Do not include any additional text or explanation in your response.

Example Input
Activity description: ‘IT hardware and software for enterprise operations’
NAICS titles, descriptions and codes:
[
{{"naics_title": "Computer Facilities Management Services","naics description": "Computer systems facilities (i.e., clients’ facilities) management and operation services","naics_code": "541513"}},
{{"naics_title": "Electronic Computer Manufacturing","naics description": "Workstations, computer, manufacturing","naics_code": "334111"}},
{{"naics_title": "Computer Facilities Management Services","naics description": "Data processing facilities (i.e., clients’ facilities) management and operation services","naics_code": "541513"}},
{{"naics_title": "All Other Support Services","naics description": "Inventory computing services","naics_code": "561990"}},
{{"naics_title": "Facilities Support Services","naics description": "Facilities (except computer operation) support services","naics_code": "561210"}},
{{"naics_title": "Computer Storage Device Manufacturing","naics description": "Storage devices, computer, manufacturing","naics_code": "334112"}},

```
{{"naics_title": "Electronic Computer Manufacturing","naics description":
"Computers manufacturing","naics_code": "334111"}},
{{"naics_title": "Computer Terminal and Other Computer Peripheral Equipment
Manufacturing","naics description": "Computer input/output equipment
manufacturing","naics_code": "334118"}},
{{"naics_title": "Electronic Computer Manufacturing","naics description":
"Computer servers manufacturing","naics_code": "334111"}}
]

### Example Output ###
[
{{‘naics_code’: 541513,
‘justification’: ‘This NAICS title covers the management and operation of
computer systems facilities and data processing facilities for clients,
which aligns with providing IT hardware and software for enterprise
operations.’,
‘naics_title’: ‘Computer Facilities Management Services’}},
{{‘naics_code’: 334111,
‘justification’: ‘This covers manufacturing of computers, servers, and
workstations, which are relevant hardware components for enterprise IT
operations.’,
‘naics_title’: ‘Electronic Computer Manufacturing’}}
]

### Input ###
Activity description: {}
NAICS titles, descriptions and codes: {}
```

Recommend the best reference products prompt

```
### Instructions ###
You are a Life Cycle Assessment expert and is performing a process-based
Life Cycle Assessment (LCA).
I will give you an ‘item description’ and a list of ‘reference_product’s.
Find a maximum of 5 highest matched ‘reference_product’s from the given list
and report them in the order of match (highest match first) in a Python list
of dictionaries.
For each entry in the dictionary, you must provide the ‘justification’ field,
‘reference_product’ and the ‘index’ of the ‘reference_product’.


### Detailed Instructions ###
- Definition of match: Given an item description, a reference product is
considered a match if it is either an exact match of the reference product
(e.g. ‘tomato’ in an exact match for item "red tomato") or one of the
components of the item description (e.g. given ‘chili bean sauce’ as the
item, and ‘bean’ and ‘chilli’ as reference products, both are considered
matches). Note that this definition is optimized for recall (i.e. to capture
as many reference products as possible in the top 5 list).
- Write the ‘justification’ field following this chain of thoughts:
(a) break down the "Item description" into its components if it has multiple
components (b) go through each reference product in the list and see if any
of them are either an exact match of the item or one of its components.
- Similar reference products are not considered as a match. Example: For the
item description ‘rapini’, the reference product ‘spinach’ is not a match;
they are similar vegetables but not the same.
- ALL ‘index’ and ‘reference_product’s that you return as an output MUST be
```

```
in the reference_product's input data. DO NOT generate or hallucinate an
'index' or a 'reference_product' that is not in the input.
- If any of the reference product is not a match, do not report that entry.
It is better to report fewer entries than to report unmatched entries, but
report at least 1 entry if possible.
- If none of reference products is a match, return one entry with
'reference_product': '', 'index': ''. DO NOT return this entry once there is
at least one match.
- If the given list contains duplicate reference products, only report the
first occurrence of the reference product.
- Always give a higher ranking to reference products that cover the most
volume (thereby contributes to more CO2 emission) of the item. e.g. given
'chili bean sauce' as the item, and 'bean' and 'chilli' as reference
products, you have to rank 'bean' higher than 'chilli' as 'chili bean sauce'
is made from beans.
- Return a python list of dictionaries, each dictionary should contain the
fields 'index', 'justification' and 'reference_product'. Output nothing else
before and after this list of dictionary.
- The format of response must match the following example:[{{'justification':
'', 'reference_product': '', 'index':}},{{'justification': '',
'reference_product': '', 'index':}}]. Do not include any additional text or
explanation in your response. DO NOT break this rule.
- Be sure to properly escape any special characters in the 'justification'
field so that your response can be parsed with a python code interpreter.


### Example Input ###
item description: "Ingredient    popped popcorn"
```

```
Reference products:
 [
 {{‘index’: 0, ‘reference_product’: ‘sweet corn’}},
 {{‘index’: 1, ‘reference_product’: ’chopping, maize’}},
 {{‘index’: 2, ‘reference_product’: ‘maize chop’}},
 {{‘index’: 3, ‘reference_product’: ‘palm kernel meal’}},
 {{‘index’: 4, ‘reference_product’: ‘maize grain, organic’}},
 {{‘index’: 5, ‘reference_product’: ‘lime, packed’}},
 {{‘index’: 6, ‘reference_product’: ‘flax husks’}},
 {{‘index’: 7, ‘reference_product’: ‘maize grain’}},
 {{‘index’: 8, ‘reference_product’: ‘wood chips and particles, willow’}},
 {{‘index’: 9, ‘reference_product’: ‘maize grain, feed’}},
 {{‘index’: 10, ‘reference_product’: ‘lime, hydrated, packed’}},
 {{‘index’: 11, ‘reference_product’: ‘maize seed, at farm’}},
 {{‘index’: 12, ‘reference_product’: ‘tomato, fresh grade’}},
 {{‘index’: 13, ‘reference_product’: ‘quicklime, milled, packed’}},
 {{‘index’: 14, ‘reference_product’: ‘bottom ash, MSWI[F]-WWT, WW from
 maize starch production’}},
 {{‘index’: 15, ‘reference_product’: ‘maize grain, feed, organic’}},
 {{‘index’: 16, ‘reference_product’: ‘frozen fish sticks, hake’}},
 {{‘index’: 17, ‘reference_product’: ‘sweet sorghum grain’}},
 {{‘index’: 18, ‘reference_product’: ‘operation, reefer, freezing’}},
 {{‘index’: 19, ‘reference_product’: ‘orange, fresh grade’}}
 ]


### Example Output ###
[
```

```
{{"justification": ‘popcorn is made from maize/corn kernels, so "maize
grain" is the most relevant reference product.’,"reference_product": "maize
grain","index": 7}},
{{"justification": ’"Maize grain, organic" is also highly relevant as
popcorn is made from maize/corn.’,"reference_product": "maize grain,
organic","index": 4}},
{{"justification": ’"Maize grain, feed" is relevant as it refers to maize/
corn which is used to make popcorn.’,"reference_product": "maize grain,
feed","index": 9}},
{{"justification": ’"Maize seed, at farm" is relevant as it refers to the
source of maize/corn used for popcorn.’,"reference_product": "maize seed, at
farm","index": 11}},
{{"justification": ’"Maize grain, feed, organic" is somewhat relevant as it
refers to organic maize/corn which could be used for
popcorn.’,"reference_product": "maize grain, feed, organic","index": 15}},
]


### Input ###
Information on item: {}
Reference products: {}
```

Recommend the best EFs prompt

```
### Instructions ###
You are a Life Cycle Assessment expert and is performing a process-based
Life Cycle Assessment (LCA).
I have (a) an ‘item description’ and (b) the name (‘impact_factor_name’) and
```

```
associated information (‘reference_product’, ‘process_technology’,
‘process_description’) of a few candidate impact factors.
Report the first and the second (in that order of match) ‘impact_factor_name’
that ‘exactly matches’ with the given ‘item description’.
Note that this task is optimized for precision (i.e. to find the exact match).
In each entry report ‘justification’, ‘impact_factor_name’, ‘index’ (in this
order). This ‘index’ value should refer to the index from the provided impact
factor list.

### Detailed Instructions ###
- Definition of exact match:
    - An ‘exact match’ happens when (a) the item description is directly
    covered by a single impact factor (e.g. ‘market for grape’ covers ‘black
    grape’ comprehensively), or (b) each components of the given item is
    covered by a single the impact factor under consideration.
    - Similar impact factors are not considered as exact match. Example: For
    the item description ‘rapini’, if you are given ‘market for spinach’ as
    a relevant impact factor (since spinach is a similar vegetable), you
    have to return None for ‘index’ and ‘impact_factor_name’ as ‘emission
    associated with spinach cannot be used to estimate the emissions
    associated with rapini’ (justification).
    - Once you breakdown an item with multiple components or ingredients,
    the exact match MUST cover ALL components and processes happened on the
    item. If there is no impact factor that covers processes happened on the
    main component, you MUST return None.
    - Examples of cases that item description has multiple components:
        - For item description ‘frozen artichoke’, ‘market for artichoke’ is
```

```
        NOT an exact match because it does not cover the process of freezing.
        - Another example of item description is ‘ground lamb’ which
        includes the process of grounding, so do not choose ‘market for
        sheep for slaughtering, live weight’.
        - For item description ‘popcorn’, an exact impact factor MUST
        include the process of popping and the corn, so the ‘market for
        maize grain, organic’ is not a exact match, so you MUST return None.
- Write the ‘justification’ field following this chain of thoughts:
    - break down the "Item description" into its components if it has
    multiple components. e.g. if given ‘chocolate milk’, write ‘chocolate
    milk consists of multiple components such as chocolate and milk’.
    Then go step by step through each impact factor from the given list
    and see if any of them comprehensively cover all the emissions
    associated with all the components of the item description.
    - if there is no exact match, return None for ‘index’ and
    ‘impact_factor_name’. Example: given ‘chocolate milk’ as item
    description and (i) ‘market for milk’, (ii) ‘milk production’ (iii)
    chocolate production as three candidate impact factors, you have to
    return None for for both ‘index’ and ‘impact_factor_name’ as none of the
    impact factors provided capture the full emissions associated with all
    two ingredients. They only capture one of the main ingredients like
    milk or chocolate.
    - If there is an exact match, report the ‘index’ and
    ‘impact_factor_name’ for that impact factor. Example, given ‘chocolate
    milk’ as item description and (i) ‘market for milk’, (ii) ‘market for
    chocolate milk’ (iii) ‘market for chocolate’ as three candidate impact
    factors, you have to return ‘market for chocolate milk’ and it’s ‘index’
```

as it exactly matches with the item description.
- Whenever there is an impact factor starting with ‘market for’ and other impact factors that has ‘production’ in their name like ‘X production’ or ‘production of X’, ALWAYS SELECT the impact factor starting with ‘market for’ as the best match. This rule is to be followed strictly, without any exceptions or considerations for additional details or qualifiers in either impact factors.
- Examples:
    - Given ‘table grape’ and both ‘market for grape’ and ‘grape production’, ‘market for grape’ MUST be chosen as the best match, and ‘grape production’ as the second-best match.
    - Given ‘tomato, fresh grade’ and both ‘market for tomato, fresh grade’ and ‘tomato production, fresh grade, open field-tomato, fresh grade’, ‘market for tomato, fresh grade’ MUST be chosen as the best match, and ‘tomato production, fresh grade, open field-tomato, fresh grade’ as the second-best match, regardless of additional details in the ‘production’ impact factor.
    - Given ‘polypropylene, granulate’ and both ‘market for polypropylene, granulate’ and ‘polypropylene production, granulate-polypropylene, granulate’, ‘market for polypropylene, granulate’ MUST be chosen as the best match, and ‘polypropylene production, granulate-polypropylene, granulate’ as the second-best match.
- You should NEVER, under any circumstances, select an impact factor that has ‘production’ in the name as the best match when there is an impact factor starting with ‘market for’ available as an option.
- If there are duplicate impact factors (exact same name), only report the first occurrence of the impact factor.

```
- If there are two similar impact factors and one of them has ‘organic’ in
the name, you must return the impact factor that does not contain ‘organic’,
unless the item description clearly indicates the item is organic.
- Return a Python list of dictionaries where each dictionary contains the
keys ‘index’, ‘impact_factor_name’, and ‘justification’. The format of each
dictionary must match the following example: [{{‘justification’: ‘’,
‘index’: , ‘impact_factor_name’: ‘’}},{{‘justification’: ‘’, ‘index’: ,
‘impact_factor_name’: ‘’}}]. Do not include any additional text or
explanation in your response.
- Be sure to properly escape any special characters in the ‘justification’
field so that your response can be parsed with a python code interpreter.
- All ‘index’ and ‘impact_factor_name’ that you return MUST be from the
input or can be None. DO not generate or hallucinate ‘index’ that is not
in the input list.

### Example Input ###
item description: "hot salsa"
Impact factor list:
{{
  0: {{‘impact_factor_name’: ‘tomato production, fresh grade, open field-
  tomato, fresh grade’,
  ‘reference_product’: ‘tomato, fresh grade’,
  ‘process_technology’: ’Cultivation of Tomato follows conventional
  production standards’,
  ‘process_description’: ‘This dataset represents the production of 1 kg of
  fresh Tomato fresh grade during December, 2015- June, 2016 at a farm
  located in Village- Bhagatpur Ratan, District- Moradabad of UP. In this
```

```
  area, fields are irrigated with ground water using electric pump for
  surface irrigation. The farm has a yield of 60,000 kg fresh matter per ha
  for this crop cycle. Sowing date for the crop is 10-Feburary, 2016 and
  harvesting date is 12-June, 2016. Mineral NP fertilizer input is 113.25-69
  kg/ha. Herbicide used has active ingredient as Carbofuran 495g/ha,
  Cypermethrin 156.25g/ha Machinery is used only for soil cultivation with
  fuel consumption of 12.48 kg/ha using plough and rotary cultivator for
  soil tillage. This dataset is meant to replace the following datasets:  -
  tomato production, fresh grade, IN-UP, 2015 - 2018 (877f8df3-fbfc-408f-
  beee-46664f92cdbb)'}},
 1: {{'impact_factor_name': 'market for coriander',
  'reference_product': 'coriander',
  'process_technology': nan,
  'process_description': 'This is a market activity. Each market represents
  the consumption mix of a product in a given geography, connecting
  suppliers with consumers of the same product in the same geographical
  area. Markets group the producers and also the imports of the product (if
  relevant) within the same geographical area. They also account for
  transport to the consumer and for the losses during that process, when
  relevant. This is the market for  coriander, in the Global geography.
  Transport from producers to consumers of this product in the geography
  covered by the market is included. The product coriander is a spice/
  aromatic crop. It is a perennial crop.'}},
}}

### Example Response ###
[
```

```
{{ 'justification': 'hot salsa contains multiple ingredients such as tomato,
chilli, corn, and none of the provided impact factors capture the emissions
associated with all the typical ingredients. They only capture one of the
main ingredients like tomato.', 'index': None, 'impact_factor_name': None}},
{{ 'justification': 'hot salsa contains multiple ingredients such as tomato,
chilli, corn, and none of the provided impact factors capture the emissions
associated with all the typical ingredients. They only capture one of the
main ingredients like tomato.', 'index': None, 'impact_factor_name': None}}
]


Input For you
=============
Item description: {}
Impact factor list:
{}
```

pLCA EF Recommendation Algorithm

Annotation Instructions

To ensure consistent annotations, we provided annotators with documentation that explains both the annotation procedure and the underlying scientific principles. The annotation process involves selecting the most appropriate EF from a list of options that best matches a given input activity description. Annotators are instructed to carefully read and understand the input text, consider AI-generated interpretations and recommendations, and make selections based on their understanding. The goal is to benchmark the performance of the Parakeet model in automating this mapping process. The instructions emphasize the im-

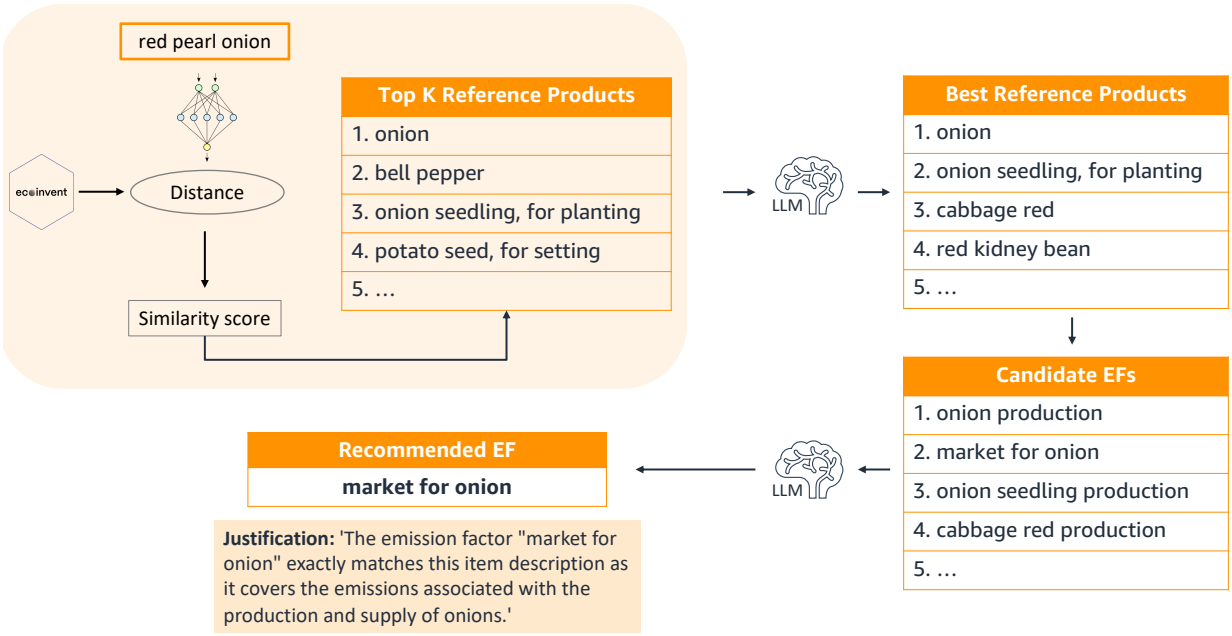


Figure 2: An example of Parakeet's pLCA EF recommendation process. First, Parakeet uses an embedding model to retrieve the most semantically similar reference products to the given activity. Then, the corresponding EFs to the retrieved reference products are extracted. Finally, these candidate EFs are fed into the LLM model, which recommends the best matched EF and generates a justification for this recommendation.

portance of identifying cases where no good match is available or when the input text is unclear.

Figure 3 shows the user interface of our annotation process. We provide annotators a detailed explanation of the annotation interface, including descriptions of the left pane (which contains the input activity and AI interpretations) and the right pane (where annotators make their selections and provide feedback). We ask them to select the best EF from the options in the right pane. We also provide them the key terminology related to Life Cycle Assessment (LCA), Environmental Impact Factors (EIFs), and economic classification systems like NAICS. The instructions include tips for annotation, examples of challenging cases, and a FAQ section to help annotators understand the context and importance of their work in calculating corporate carbon footprints.



Figure 3: Annotation interface for mapping an EEIO input activity to the best economic sector.

References

(1) Ingwersen, W. W.; Li, M.; Young, B.; Vendries, J.; Birney, C. USEEIO v2. 0, the US environmentally-extended input-output model v2. 0. *Scientific Data* **2022**, *9*, 194.

(2) United States Census Bureau 2022 NAICS Definition. 2022; `https://www.census.gov/naics/reference_files_tools/2022_NAICS_Manual.pdf`.

(3) Balaji, B.; Vunnava, V. S. G.; Guest, G.; Kramer, J. CaML: Carbon footprinting of household products with zero-shot semantic text similarity. Proceedings of the ACM Web Conference 2023. 2023; pp 4004–4014.

(4) Wernet, G.; Bauer, C.; Steubing, B.; Reinhard, J.; Moreno-Ruiz, E.; Weidema, B. The ecoinvent database version 3 (part I): overview and methodology. *The International Journal of Life Cycle Assessment* **2016**, *21*, 1218–1230.

(5) Sun, Z.; Chen, M.; Hu, W. Knowing the No-match: Entity Alignment with Dangling Cases. Proceedings of the 59th Annual Meeting of the Association for Computational

Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). 2021; pp 3582–3593.

(6) Balaji, B.; Vunnava, V. S. G.; Domingo, N.; Gupta, S.; Gupta, H.; Guest, G.; Srinivasan, A. Flamingo: Environmental Impact Factor Matching for Life Cycle Assessment with Zero-shot Machine Learning. *ACM Journal on Computing and Sustainable Societies* **2023**, *1*, 1–23.

(7) Sphera GaBi Product Sustainability Software. 2024; `https://sphera.com/solutions/product-stewardship/life-cycle-assessment-software-and-data/`.

(8) Luo, B.; Liu, J.; Deng, Z.; Yuan, C.; Yang, Q.; Xiao, L.; Xie, Y.; Zhou, F.; Zhou, W.; Liu, Z. AutoPCF: A Novel Automatic Product Carbon Footprint Estimation Framework Based on Large Language Models. Proceedings of the AAAI Symposium Series. 2023; pp 102–106.

(9) Meinrenken, C. J.; Kaufman, S. M.; Ramesh, S.; Lackner, K. S. Fast carbon footprinting for large product portfolios. *Journal of Industrial Ecology* **2012**, *16*, 669–679.

(10) Anthropic Claude 3.5 Sonnet. 2024; `https://www.anthropic.com/news/claude-3-5-sonnet`.

(11) Liu, X.; Zheng, Y.; Du, Z.; Ding, M.; Qian, Y.; Yang, Z.; Tang, J. GPT understands, too. *AI Open* **2023**, *5*, 208–215.

(12) Muennighoff, N.; Tazi, N.; Magne, L.; Reimers, N. MTEB: Massive Text Embedding Benchmark. Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics. 2023; pp 2014–2037.

(13) Stadler, K.; Wood, R.; Bulavskaya, T.; Södersten, C.-J.; Simas, M.; Schmidt, S.; Usubiaga, A.; Acosta-Fernández, J.; Kuenen, J.; Bruckner, M.; others EXIOBASE 3: Devel-

oping a time series of detailed environmentally extended multi-regional input-output tables. *Journal of Industrial Ecology* **2018**, *22*, 502–515.

(14) NTNU EXIOBASE concordances. 2025; `https://ntnu.app.box.com/v/EXIOBASEconcordances`.